

A PROPOSED DESIGN AND IMPLEMENTATION

DSN4091 – CAPSTONE PROJECT PHASE-I

Phase – I Report

Submitted by

KISLAY ANAND (23BAI10359)

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)



VIT[®]
BHOPAL
www.vitbhopal.ac.in

SCHOOL OF COMPUTING SCIENCE AND ENGINEERING

VIT BHOPAL UNIVERSITY, SEHORE,

MADHYA PRADESH – 466114

September 2026



BONAFIDE CERTIFICATE

Certified that this project report titled “**A PROPOSED DESIGN AND IMPLEMENTATION FOR AUTOINSPECT-X: CNN-TRANSFORMER HYBRID DEEP LEARNING FOR VEHICLE DAMAGE SEGMENTATION**” is the bonafide work of “**KISLAY ANAND (23BAI10359)**” who carried out the project work (DSN4091 – Capstone Project Phase-I) under my supervision. Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

PROGRAM CHAIR

Dr. Pradeep Kumar Mishra

Senior Assistant Professor (Gr-2)

School of Computing Science Engineering and
Artificial Intelligence

VIT BHOPAL UNIVERSITY

PROJECT GUIDE

Dr. Rudra Kalyan Nayak

Senior Assistant Professor (Gr-2)

School of Computing Science Engineering and
Artificial Intelligence

VIT BHOPAL UNIVERSITY

The DSN4091 – Capstone Project Phase-I Viva Voce Examination is held on _____

ACKNOWLEDGEMENT

First and foremost, I would like to thank the Lord Almighty for His presence and immense blessings throughout the project work.

I would like to thank my internal guide Dr. Rudra Kalyan Nayak, for continually guiding and actively participating in my project, and giving valuable suggestions to complete the project work.

I wish to express my heartfelt gratitude to Dr. Pradeep Kumar Mishra, PC-Lead, School of Computing Science Engineering and Artificial Intelligence, for much of his valuable support and encouragement in carrying out this work.

I wish to express my heartfelt gratitude to Dr. Pon Harshavardhanan, Dean, School of Computing Science Engineering and Artificial Intelligence, for much of his valuable support and encouragement in carrying out this work.

I would like to thank all the technical and teaching staff of the School of Computing Science Engineering and Artificial Intelligence, who extended directly or indirectly all support.

Last, but not least, I am deeply indebted to my parents who have been the greatest support while I worked day and night for the project to make it a success.

ABSTRACT

Vehicle-exterior damage inspection is still largely a manual, judgement-based activity. Insurance claims, rental-car returns and garage intake rely on a human looking at photographs and describing what is visible; many automated tools overreach by presenting uncalibrated model output as measured ground truth, or by inventing repair-cost figures that cannot be derived from an uncontrolled photograph. This project develops AutoInspect-X, a photo-first vehicle damage inspection system that keeps model output honest. A single photograph of the damaged area is first validated by a capture-quality gate (luminance, glare, blur and contrast heuristics); when it passes, a deep segmentation network predicts per-pixel damage classes; and an evidence module derives transparent, image-denominator area ratios and confidence values that are explicitly labelled as model predictions, never as verified physical measurements. The work is grounded in the CarDD dataset (4,000 images with official train/validation/test splits), and compares a CNN+Transformer hybrid segmentation model (ResNet-34 encoder, bottleneck transformer and U-Net-style decoder, deep-supervised Dice+Focal training) against a ResNet-34 U-Net baseline. The system itself comprises a FastAPI backend, a LangChain/LangGraph conversational assistant (Groq-backed online, deterministic offline), an opt-in consent pipeline that stores training samples only with explicit user agreement, and a Next.js/React demonstration front end. This Phase-I report records the analysis: problem definition, literature study, requirement artefacts, design methodology, implementation, measured pilot results, a walk-through of validation scenarios, and the limitations and future scope. Repair costs and pricing are explicitly out of scope. The 15-epoch pilot checkpoints reached a validation foreground mIoU of about 0.60–0.61 and are documented as intermediate evidence, not as a final scientific conclusion.

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
Fig. 1.1	Photo-first inspection workflow of AutoInspect-X.	1
Fig. 4.1	Software architecture of the AutoInspect-X demo system.	10
Fig. 5.1	First frame of the scroll-driven cinematic home-page intro (frame 1/913).	14
Fig. 5.2	Near-end frame of the cinematic intro sequence (frame 913/913).	14
Fig. 5.3	Inspection composer at /demo with upload prompt and free-text field.	15
Fig. 5.4	Blurred input photograph used to exercise the capture-quality gate.	15
Fig. 5.5	Capture-quality rejection (TOO_BLURRY) with retake guidance; no mask is produced.	16
Fig. 5.6	Minor-scratch test photograph (street scene).	17
Fig. 5.7	Minor-scratch result - detected region, mean confidence, image-area fraction, consent prompt.	18
Fig. 5.8	Minor dent/scratch test photograph (close-up).	19
Fig. 5.9	Minor-damage result - scratch chip with confidence, damage-area ratio, consent granted.	20
Fig. 5.10	Medium-damage test photograph.	21
Fig. 5.11	Medium-damage result - dent, scratch and lamp-broken chips with consent prompt.	22
Fig. 5.12	Major-damage test photograph.	22
Fig. 5.13	Major-damage result - dent with a small glass-shatter region and consent prompt.	23
Fig. 5.14	Multi-cue (confusing) test photograph.	24
Fig. 5.15	Multi-cue result - dent, crack and tire-flat chips with mean confidence.	25
Fig. A.1	Composer with the photo attached before analysis (minor-scratch session).	31
Fig. A.2	Assistant narrative for a minor dent (the assistant's "Here is	32

	what the visual model identified” summary).	
Fig. A.3	Assistant narrative for a medium-damage photograph.	33
Fig. A.4	Major-damage result with consent granted and re-analysis confirmation.	34
Fig. A.5	Assistant narrative for the multi-cue (confusing) photograph.	35

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	LIST OF FIGURES	v
	ACKNOWLEDGEMENT	iii
	ABSTRACT	iv
CHAPTER 1	PROJECT DESCRIPTION AND OUTLINE	1
	1.1 INTRODUCTION	1
	1.2 MOTIVATION FOR THE WORK	1
	1.3 PROBLEM STATEMENT	2
	1.4 OBJECTIVE OF THE WORK	2
	1.5 SUMMARY	3
CHAPTER 2	RELATED WORK INVESTIGATION	4
	2.1 EXISTING APPROACHES/METHODS	4
	2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS	4
	2.3 SUMMARY	5
CHAPTER 3	REQUIREMENT ARTIFACTS	6
	3.1 INTRODUCTION	6
	3.2 HARDWARE AND SOFTWARE REQUIREMENTS	6
	3.3 SPECIFIC PROJECT REQUIREMENTS	6
	3.4 SUMMARY	8
CHAPTER 4	DESIGN METHODOLOGY AND ITS NOVELTY	9
	4.1 METHODOLOGY AND GOAL	9
	4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS	9
	4.3 SOFTWARE ARCHITECTURAL DESIGNS	10
	4.4 USER INTERFACE DESIGNS	11
	4.5 SUMMARY	11
CHAPTER 5	TECHNICAL IMPLEMENTATION AND ANALYSIS	12
	5.1 OUTLINE	12
	5.2 TECHNICAL CODING AND CODE SOLUTIONS	12
	5.3 PROTOTYPE SUBMISSION	13
	5.4 SUMMARY	26
CHAPTER 6	PROJECT OUTCOME AND APPLICABILITY	27
	6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM	27
	6.2 SIGNIFICANT PROJECT OUTCOMES	27
	6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS	28
	6.4 INFERENCE	28
CHAPTER 7	CONCLUSIONS AND RECOMMENDATION	29
	7.1 OUTLINE	29
	7.2 LIMITATION/CONSTRAINTS OF THE SYSTEM	29

	7.3 FUTURE ENHANCEMENTS	29
	7.4 INFERENCE	29
	APPENDIX A - SCREENSHOTS	31
	APPENDIX B - CODE LISTINGS	36
	REFERENCES	41

CHAPTER 1 PROJECT DESCRIPTION AND OUTLINE

1.1 INTRODUCTION

Vehicle damage claims, rental-car returns and garage intake all begin the same way: someone takes photographs and someone else reads them. The photographs are uncontrolled – different distances, angles, lighting and compression – and the reading is manual and subjective. Automated damage-inspection products promise to fix the second problem but usually inherit the first one, and many present model output as if it were a measured fact, occasionally adding confident-looking cost figures that no photograph can justify.

AutoInspect-X takes a deliberately photo-first and evidence-labelled approach. The system accepts one photograph of a damaged area, rejects photograph-quality problems that would make any interpretation unreliable (dark, glaring, blurry or low-contrast shots), and only then runs a deep segmentation model that predicts per-pixel damage classes. From that mask the system derives image-denominator area ratios and confidence values that are always labelled as model predictions. A conversational assistant explains the findings from the structured evidence and refuses to invent severity, repair actions or costs. If the user agrees, the anonymised sample can be kept for future training.

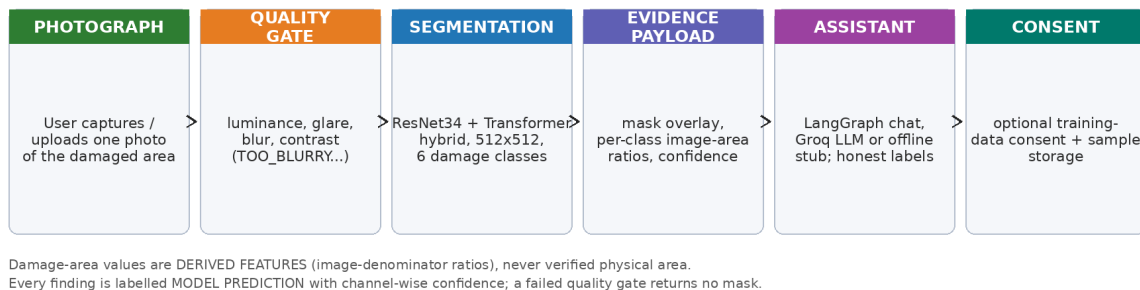


Fig. 1.1 Photo-first inspection workflow of AutoInspect-X.

Figure 1.1 summarises the workflow: capture, quality gate, segmentation, evidence payload, assistant and optional consent. The design rests on three integrity rules carried throughout the repository: real ground truth, synthetic labels, derived features and model predictions are kept in distinct labelled categories; area ratios are normalised by the image, never claimed as physical cm²; and any cost or repair prediction is out of scope (recorded in the project's decision log, ADR 0011).

1.2 MOTIVATION FOR THE WORK

The motivation for this work comes from three observed gaps in how vehicle damage is handled today. First, inspection is manual: a claims adjuster or a workshop mechanic spends time looking at photos that are often of poor quality, and the resulting written report is hard to reproduce. Second, automation has an honesty problem: off-the-shelf ‘damage detectors’ routinely quote damage percentages or repair costs that they cannot actually measure, and even academic tools rarely separate what a model predicted from what a human verified. Third, the data that would let a model improve is locked in customer photos, and collecting it without making the user explicitly consent is both legally fragile and ethically questionable.

A photo-first system addresses all three. A quality gate removes the inputs that make any automated reading unreliable before the model is ever invoked. Honest labels – image-denominator area, channel-wise confidence, low-confidence flags and model notes – let a human interpreter decide how much to trust the output. And an explicit consent pipeline turns everyday inspections, with the owner’s permission, into labelled training data. This project is therefore as much about the conversational and consent infrastructure around a segmentation model as about the model itself.

1.3 PROBLEM STATEMENT

Manual vehicle-damage documentation from photographs is time-consuming, subjective and hard to scale, while automated alternatives frequently present uncalibrated predictions – including fabricated cost figures – as ground truth, and typically ignore that many damaged-car photographs are too dark, blurry or glaring to be interpreted reliably at all. Formally, the problem addressed here is: given a single uncontrolled photograph of a damaged vehicle exterior, how can a system (i) decide whether the photograph is usable at all, (ii) produce a per-class damage segmentation over the CarDD damage categories, (iii) report the result as transparent, evidence-labelled features rather than as verified measurements, and (iv) explain the finding conversationally while refusing to fabricate severity, repairs or costs, and without using any user photograph without explicit consent?

A further subtlety is that legitimate damage-area ground truth in cm^2 cannot be measured from an uncontrolled photo without geometric calibration. The system therefore reports damage-area as a fraction of the image, which is a derived feature and is communicated as such. This keeps the science honest while still giving an adjuster a reproducible number.

1.4 OBJECTIVE OF THE WORK

The aim of the project is to design, implement and demonstrate a photo-first vehicle damage inspection system with evidence-labelled confidence. The specific objectives are:

1. Acquire and audit the CarDD-COCO dataset, preserve its official train/validation/test splits, and document its class and split statistics.
2. Design and implement two segmentation architectures: a ResNet-34 U-Net baseline and a CNN+Transformer hybrid (bottleneck transformer with additive fusion).
3. Implement a production-grade training harness (deep-supervised Dice+Focal loss, class-aware sampling, EMA, cosine schedule, mixed precision) and record reproducible pilot runs.
4. Build a capture-quality gate using luminance, glare, blur and contrast heuristics so that unusable photographs are rejected before model inference, with actionable retake guidance.
5. Extract instance-level damage features from the predicted mask (connected components, image-denominator area ratios and confidence), always labelled as model predictions.
6. Implement the product layer around the model: a FastAPI REST backend, a LangChain/LangGraph conversational assistant (Groq-backed online, deterministic offline stub), a Next.js/React demonstration front end, and SQLite storage.
7. Provide an explicit, optional training-data consent flow that stores anonymised samples only when the user agrees.
8. Keep quality gates green (ruff, mypy strict, pytest, Playwright) and record limitations honestly in every model reply.

1.5 SUMMARY

This chapter introduced the photo-first philosophy, the motivation, and the problem and objectives that structure the rest of the report. Chapter 2 reviews datasets and segmentation approaches relevant to vehicle damage inspection. Chapter 3 collects the requirement artefacts. Chapter 4 presents the design methodology, architecture, models and the integrity rules. Chapter 5 documents the technical implementation with real code excerpts and the demonstration walk-through. Chapter 6 reports the pilot outcomes and applicability, and Chapter 7 concludes with limitations and future work.

CHAPTER 2 RELATED WORK INVESTIGATION

2.1 EXISTING APPROACHES/METHODS

The literature splits naturally into datasets, segmentation architectures and whole damage-detection systems. On the dataset side, CarDD is the key public resource: 4,000 vehicle-damage images with more than 9,000 surface-damage instances across dent, scratch, crack, glass shatter, lamp broken, and tire flat categories, distributed with official train/validation/test splits [Wang et al., IEEE TITS 2023]. VehiDE provides a second, differently collected damage set used here as a descriptive spectrum for training-time mixing, and CrashCar101 contributes curated real-world vehicle photos used as visual testing exemplars; neither supplies validated physical area measurements [Huynh et al., KSE 2023; Parslöv et al., WACV 2024].

On the architecture side, U-Net established the encoder–decoder-with-skip segmentation pattern that remains effective with small datasets [Ronneberger et al., 2015], while ResNet provides a strong, image-pretrained feature backbone [He et al., 2016]. Vision transformers brought explicit global context that convolutional receptive fields lack but are known to be data-hungry [Dosovitskiy et al., 2021]. Damage-specific hybrid work – for example CDA-Net, which couples attention with depth cues – shows that hybrid designs help for vehicles, though it depends on additional sensor input [Bhogale et al., 2023]. Instance imbalance in damage data is typically addressed with Focal loss [Lin et al., 2017], which this project combines with a soft-Dice objective.

On the system side, the literature is dominated by classification of damage type and rough severity from a single network, with almost no work exposing (i) a capture-quality gate, (ii) explicitly image-denominator area with a stated non-physicality caveat, or (iii) a conversational, evidence-grounded assistant. LLM chat assistants applied to inspection are promising but hallucinate unless their prompt is grounded in structured evidence and forbids repair/cost invention – exactly the contract implemented by the assistant service in this project.

2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS

Table 2.1 summarises the strengths and limitations of the approaches reviewed.

Approach / work	Strengths	Limitations
CarDD dataset (2023)	Large, dedicated vehicle-damage dataset; COCO instance masks; damage-type taxonomy matches this project.	Uncontrolled photos; no validated physical-area annotations; near-duplicate images need leakage care.
VehiDE (2023); CrashCar101 (2024)	Additional damage spectra; useful for training-time mixing and visual exemplars.	Different taxonomies; neither validates cm ² ground truth.
U-Net (2015)	Strong with limited data; elegant skip connections.	Local receptive field; limited global context.
ResNet backbone (2016)	Robust image-pretrained features; strong transfer.	Fixed receptive field without attention; heavy for small VRAM devices.
Vision Transformer (2021)	Explicit global context.	Data-hungry; less sample-efficient on small damage sets.
CDA-Net (2023)	Hybrid attention + depth improves car damage cues.	Requires depth, unavailable for single consumer photos.
Focal loss (2017)	Targets class imbalance (imbalance is severe in CarDD).	Needs tuning with Dice; gamma/weight interaction.
Rule-based cost estimation	Cheap, plausible-looking output.	Not repair ground truth; cannot be derived from one photo; removed from scope (ADR 0011).
LLM chat assistants	Natural conversational explanations.	Hallucination risk unless grounded strictly in evidence; must forbid cost invention.

2.3 SUMMARY

The review justifies the design choices in this project: CarDD for a well-defined six-class damage taxonomy with official splits; a U-Net-style decoder on a ResNet-34 backbone for the baseline; a bottleneck transformer fused additively for global context; Focal+Dice training under class-aware sampling; and a strictly evidence-grounded assistant so that conversational convenience never becomes fabrication.

CHAPTER 3 REQUIREMENT ARTIFACTS

3.1 INTRODUCTION

This chapter collects the functional, data, performance, security and look-and-feel requirements distilled from the problem statement and the objectives, together with the hardware and software environment in which the system has been developed and can run.

3.2 HARDWARE AND SOFTWARE REQUIREMENTS

Table 3.1 lists the reference hardware used for development and the minimum configuration needed to run the demonstration locally.

Component	Reference (used)	Minimum
Processor	Intel Core i5-12500H	Any x86-64 with 4+ cores
Memory	16 GB DDR4	8 GB
GPU	NVIDIA RTX 3050 Laptop, 4 GB VRAM (CUDA 12.1)	CPU-only works, slower
Storage	SSD (CarDD images $\approx$ 20 GB + checkpoints)	$\geq$ 40 GB free
Camera	any phone/DSLR (demo photographs)	any

Table 3.2 summarises the software toolchain, all open source.

Layer	Tools/versions
Language	Python 3.12 (conda env “ai”), Node.js 22.22.2 / npm 10.9.7, TypeScript 5
ML	PyTorch 2.5.1, torchvision 0.20.1, OpenCV, NumPy, Pillow (CPU build in CI)
Backend	FastAPI 0.136, uvicorn, Pydantic v2, standard-library sqlite3
Agent	LangChain 1.2, LangGraph 1.1, langchain-groq 1.1 (ChatGroq), deterministic StubAssistant offline
Frontend	Next.js 16.3.4, React 19.2.8, Tailwind CSS 4
Quality	pytest (strict markers), ruff, mypy (strict), Playwright 1.63, GitHub Actions
Tooling	uv 0.11.11, git (conventional commits)

3.3 SPECIFIC PROJECT REQUIREMENTS

3.3.1 DATA REQUIREMENT

CarDD-COCO is used with its official splits preserved and measured locally on disk: the train split holds 2,816 images with 6,211 annotations, the validation split 810 images with 1,744 annotations, and the test split 374 images with 785 annotations (4,000 images in total). All four thousand images carry annotations. The train-split class distribution is heavily imbalanced – scratch (2,560 instances), dent (1,806), crack (651), lamp broken (494), glass shatter (475) and tire flat (225) – which drives the class-aware sampling and focal weighting. Split-scoped dataset adapters guarantee train/validation/test are never mixed, and image-level near-duplicates are documented in the repository audit so that leakage can be avoided.

3.3.2 FUNCTIONS REQUIREMENT

Session lifecycle: create an inspection session, upload one photo, analyse it, query its state, and delete it.

Capture-quality gating: reject dark, glaring, blurry or low-contrast photos with retake guidance before inference.

Damage segmentation: per-pixel prediction of the six CarDD damage classes plus background.

Evidence payload: predicted mask overlay, per-class and overall image-denominator areas, per-instance confidence.

Conversational explanation: the assistant explains findings from evidence and answers follow-ups; no cost/repair output.

Training-data consent: record GRANTED/DECLINED; store an anonymised sample only when granted (MODEL_SUGGESTED provenance).

Ephemeral cleanup: session photos and state are deleted when the session closes or expires; consented samples live separately.

3.3.3 PERFORMANCE AND SECURITY REQUIREMENTS

Performance: 512×512 inference on the 4 GB RTX 3050 (mixed precision) in about a second of black-box time; asynchronous chat turns.

Accuracy gate: the assistant repeats only structured evidence and flags low-confidence results as preliminary; every reply carries model notes.

Security: session and asset ids are whitelisted (never used unsanitised in paths); no secret keys are committed; the Groq key lives only in the local environment file, while CI runs the offline stub.

Data integrity: ground-truth categories, synthetic labels, derived features and predictions are kept distinct in code, storage and UI; area ratios are image-denominator only.

Privacy: chat history and personal identifiers are never stored for training; consented samples isolate the training directory from session cleanup.

3.3.4 LOOK AND FEEL REQUIREMENTS

The demonstration front end presents a dark, cinematic brand (a scroll-driven intro rendered from a 913-frame 30 FPS sequence) that transitions into a light, conversational inspector at /demo. The composer accepts a photo (drag, tap or capture) and a free-text message, then renders the assistant reply with model chips (damage classes), confidence, image-area fraction and an explicit “Demonstration build. Findings are machine predictions with explicit labels – never verified damage” disclaimer under every result. Consent is a clearly optional step. Responsiveness is validated with Playwright over desktop, tablet and mobile viewports.

3.4 SUMMARY

Together these requirements define a minimal, honest, photo-first system: a session API around a quality gate, a segmentation engine, an evidence pipeline, an assistant, and an opt-in consent store, with a demonstration UI and CI quality gates.

CHAPTER 4 DESIGN METHODOLOGY AND ITS NOVELTY

4.1 METHODOLOGY AND GOAL

The project follows a phased, task-oriented methodology. Research phases 0–12 covered environment discovery, dataset audit and preparation, model framework selection, a smoke trainer, a full training harness, locked experiments, error analysis and the eventual architecture-specification revision. Product phases A–R covered, in order, repository audit, backend foundations, storage, the inference core, features and quality, the API wiring, the demonstration front end, the assistant/consent pipeline, end-to-end and real-engine journeys, Playwright suites, continuous integration and documentation. Each phase lands on a green quality gate (ruff, mypy strict, pytest), and material scoping changes are recorded as decision records – the most relevant being ADR 0010 (hybrid model lineage) and ADR 0011 (photo-first scope; cost, repair and questionnaire features removed).

The methodological goal is a system that is honest by construction: every stage carries labels that prevent model output from masquerading as ground truth, and the measurement conventions (image-denominator area, explicit preliminary flags) are enforced by tests.

4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS

Capture-quality module: computes sharpness (variance of Laplacian), mean luminance, bright fraction and contrast standard-deviation; maps them to VALID/TOO_DARK/EXCESSIVE_GLARE/TOO_BLURRY/INSUFFICIENT_CONTEXT; produces retake guidance, and never lets a rejected photo reach the model.

Segmentation engine: loads a checkpoint through a strict artefact contract (model_arch key, state dict), runs the training-time preprocess (512×512, 1/255, CHW), predicts 7-channel logits, and normalises deep-supervision outputs to the main head.

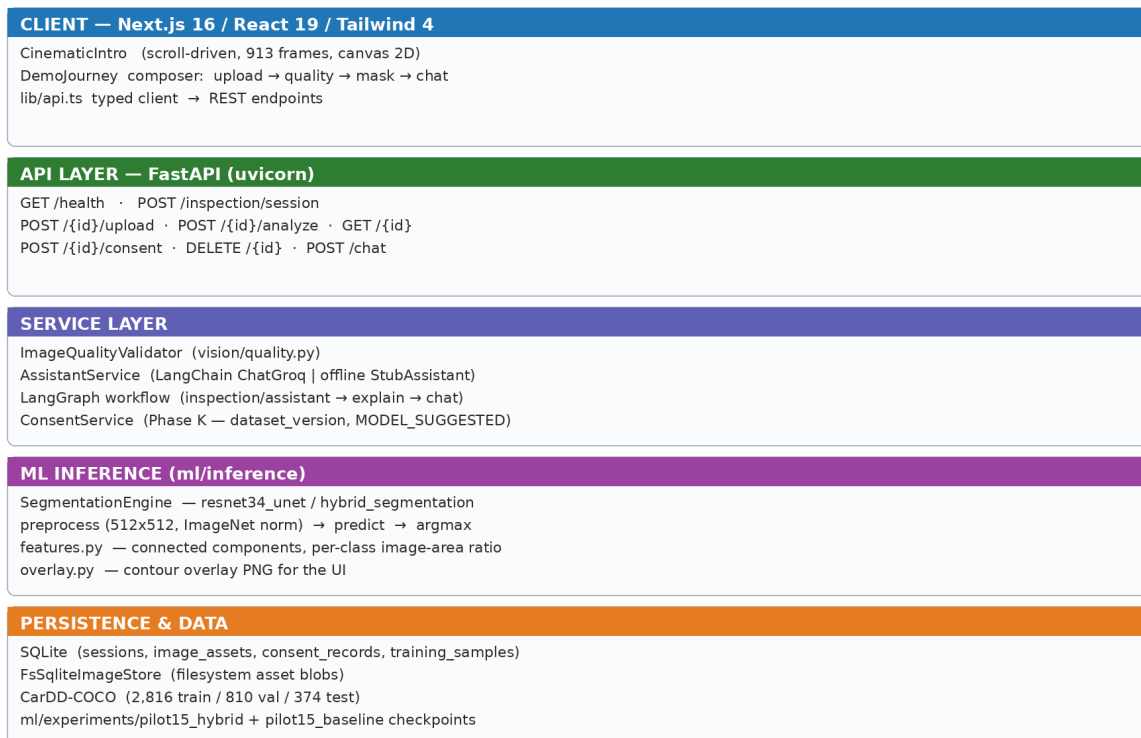
Feature extraction: runs connected components per damage class over the argmax mask, filters tiny components, and computes image-denominator area ratios, bounding boxes, centroids and per-instance confidence.

Assistant service: a protocol seam between the routers and either LangChain ChatGroq or the deterministic StubAssistant; grounded strictly in the prompt-safe evidence payload; forbids severity/cost invention.

Storage & consent service: SQLite session/store tables plus a filesystem image store; consent decisions recorded separately; training samples written with dataset version and MODEL_SUGGESTED provenance only on GRANTED.

4.3 SOFTWARE ARCHITECTURAL DESIGNS

Figure 4.1 shows the layered architecture. The Next.js client talks only to the FastAPI router set (/health, /inspection/*, /chat). Services (quality validator, assistant, LangGraph workflow, consent) sit behind the routers, the segmentation and feature code is isolated under ml/inference, and persistence is split between SQLite and the filesystem image store. The ML training code (ml/training, ml/models) is deliberately not importable from production inference (enforced and tested), so the demo cannot accidentally retrain or use training-only transformations.



Arrows represent request/response flow: page → REST → services → ML inference → storage (SQLite + filesystem).

Fig. 4.1 Software architecture of the AutoInspect-X demo system.

The REST contract is summarised in Table 4.1.

Method & path	Behaviour
GET /health	liveness + model-version metadata
POST /inspection/session	create inspection session (uuid)
POST /inspection/{id}/upload	store one photo as an image asset

POST /inspection/{id}/analyze	quality gate; segmentation; evidence; assistant reply
GET /inspection/{id}	current state (messages, quality, inspection)
POST /inspection/{id}/consent	record GRANTED/DECLINED; store sample when granted
DELETE /inspection/{id}	close session; ephemeral cleanup
POST /chat	LangGraph START → llm_turn → END; evidence-grounded reply

4.4 USER INTERFACE DESIGNS

The user interface design follows the photo-first narrative: the home page opens with a scroll-driven cinematic sequence that dramatises the inspection journey, then hands over to the composer, where the user attaches a single photo (optionally with a message). The conversation thread renders each analysis as an assistant message, enriched with damage-class chips, mean confidence, image-area fraction and the mandatory “machine predictions, not verified damage” disclaimer. Failures (quality rejection) and successes produce visibly different UI states, and training-consent is offered in-line and always optional. Chapter 5 reproduces the implemented screens.

4.5 SUMMARY

The design separates what is measured from what is inferred at every layer, keeps the inference path dependent on training code or data only through locked, tested equivalence, and makes honesty (labels, disclaimers, consent) a first-class functional requirement rather than a documentation afterthought.

CHAPTER 5 TECHNICAL IMPLEMENTATION AND ANALYSIS

5.1 OUTLINE

This chapter describes how each design element was implemented. Code excerpts are reproduced from the repository and every listing is labelled with its source file. The final section walks through the demonstration from the supplied screenshots, with the on-screen values reproduced exactly.

5.2 TECHNICAL CODING AND CODE SOLUTIONS

5.2.1 Inference preprocessing (*ml/inference/preprocess.py*)

Inference must reproduce the training transform bit-for-bit; ADR 0003 pins this equivalence and a test recomputes the training ops independently. Listing 5.1 shows the decode and preprocess path (source: *ml/inference/preprocess.py*).

Listing 5.1 – source: ml/inference/preprocess.py (abridged)

```
def load_image_rgb(source: Path | bytes) -> np.ndarray:
    """Decode a JPEG/PNG/WebP image as RGB uint8 ``HxWx3``."""
    ...
def preprocess_image(rgb: np.ndarray) -> torch.Tensor:
    if rgb.dtype != np.uint8:
        raise ValueError(f"preprocess expects uint8 RGB, got dtype {rgb.dtype}...")
    resized = cv2.resize(rgb, (TARGET_SIZE, TARGET_SIZE),
                          interpolation=cv2.INTER_LINEAR)
    normalized = resized.astype(np.float32) / 255.0
    return torch.from_numpy(np.transpose(normalized, (2, 0, 1))).contiguous()
```

5.2.2 Capture-quality gate (*apps/api/vision/quality.py*)

Listing 5.2 is the whole heuristic validator. It runs before any model call and never inspects model output; rejected photos return only guidance.

Listing 5.2 – source: apps/api/vision/quality.py (abridged)

```
class QualityStatus(StrEnum):
    VALID = "VALID"; TOO_BLURRY = "TOO_BLURRY"; TOO_DARK = "TOO_DARK"
    EXCESSIVE_GLARE = "EXCESSIVE_GLARE"; ...

def assess(self, rgb: np.ndarray) -> QualityResult:
    gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
    sharpness = float(cv2.Laplacian(gray, cv2.CV_64F).var())
    metrics = {
        "sharpness_variance": round(sharpness, 2),
        "mean_luminance": round(float(gray.mean()), 2),
        "bright_fraction": round(float((gray > 250).mean()), 4),
        "contrast_std": round(float(gray.std()), 2),
    }
    if metrics["mean_luminance"] < 45.0:
        status = TOO_DARK
    elif metrics["bright_fraction"] > 0.30:
        status = EXCESSIVE_GLARE
    elif sharpness < 25.0:
        status = TOO_BLURRY
    elif metrics["contrast_std"] < 12.0:
        status = INSUFFICIENT_CONTEXT
    ...
```

5.2.3 Analysis endpoint (apps/api/routers/inspection.py)

Listing 5.3 shows the core of the /analyze flow: quality gate first, then engine inference, then feature extraction and evidence persistence (source: apps/api/routers/inspection.py, abridged; helper names preserved).

Listing 5.3 – source: apps/api/routers/inspection.py (abridged)

```
rgb = load_image_rgb(data)
quality = ImageQualityValidator().assess(rgb)
if not quality.is_valid:
    guidance, fallback = _llm_retake_guidance(c, quality.reasons)
    ... # state updated; returns status="QUALITY_FAILED" – no mask is produc...
result = engine.predict_bytes(data)
features = extract_features(result)
overlay_png = encode_png(render_overlay(rgb, result.mask))
inspection = features.to_dict()
inspection.update({
    "mean_confidence": round(result.mean_confidence, 4),
    "damage_fraction": round(result.damage_fraction, 6),
    "low_confidence": result.quality.low_confidence,
    "model_notes": result.quality.notes,
    "quality": quality_dict,
    "model_metadata": result.metadata.to_dict(),
})
evidence = persist_evidence({"inspection": inspection})
explanation, fallback = _llm_explanation(c, evidence or {})
... # state saved; AnalyzeResponse(status="OK", ...)
```

5.2.4 Front-end photo handling (apps/web/components/demo/DemoJourney.tsx)

On the client, the composer uploads the photo and immediately calls /analyze; the reply is rendered as an assistant message with overlay and honest labels (source: apps/web/components/demo/DemoJourney.tsx, abridged).

Listing 5.4 – source: apps/web/components/demo/DemoJourney.tsx (abridged)

```
const handlePhoto = async (text: string, file: File) => {
    await uploadPhoto(sessionId, file);
    const res = await analyzePhoto(sessionId);
    const assistant: ChatMessage = res.status === "OK"
        ? { role: "assistant", content: res.assistant_message,
            overlay_png_base64: res.overlay_png_base64,
            quality_status: res.quality_status,
            classes_present: res.classes_present,
            mean_confidence: res.mean_confidence,
            damage_fraction: res.damage_fraction }
        : { role: "assistant", content: res.assistant_message,
            quality_status: res.quality_status,
            quality_reasons: res.quality_reasons };
    setMessages((m) => [...m, assistant]);
};
```

5.3 PROTOTYPE SUBMISSION

The working prototype was exercised end-to-end with the six photographs supplied for this report. Figures 5.1–5.3 show the home page and composer; Figures 5.4–5.5 show the quality gate rejecting a blurred photo; Figures 5.6–5.15 show the five analysis scenes. Every

screenshot below is a real browser capture of the running system, and the numbers reported are the ones the assistant actually produced.



Fig. 5.1 First frame of the scroll-driven cinematic home-page intro (frame 1/913).

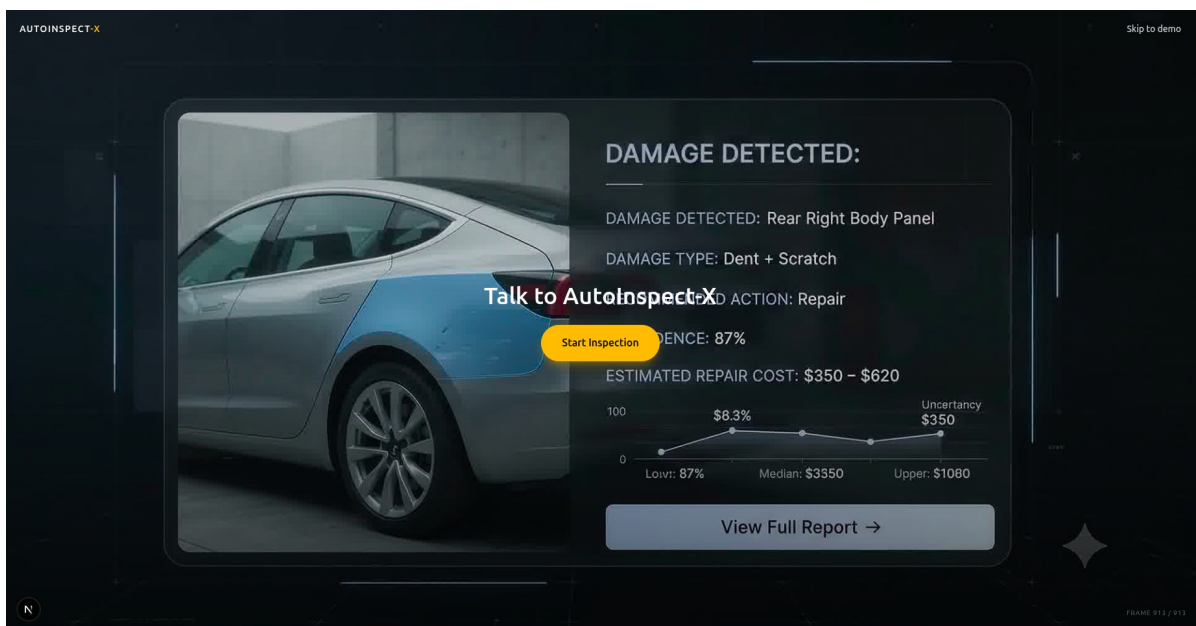


Fig. 5.2 Near-end frame of the cinematic intro sequence (frame 913/913).

The cinematic intro scrolls a pre-rendered 913-frame 30 FPS image sequence drawn to a single high-DPI canvas (frame 1/913 in Fig. 5.1, frame 913/913 in Fig. 5.2), with a prefers-reduced-motion static fallback. One narration frame of this older sequence still shows a legacy repair-cost line; that copy predates ADR 0011 and is flagged in the repository for visual reconciliation – the shipped analysis flow itself never emits cost fields. Figure 5.3 shows the composer that follows.

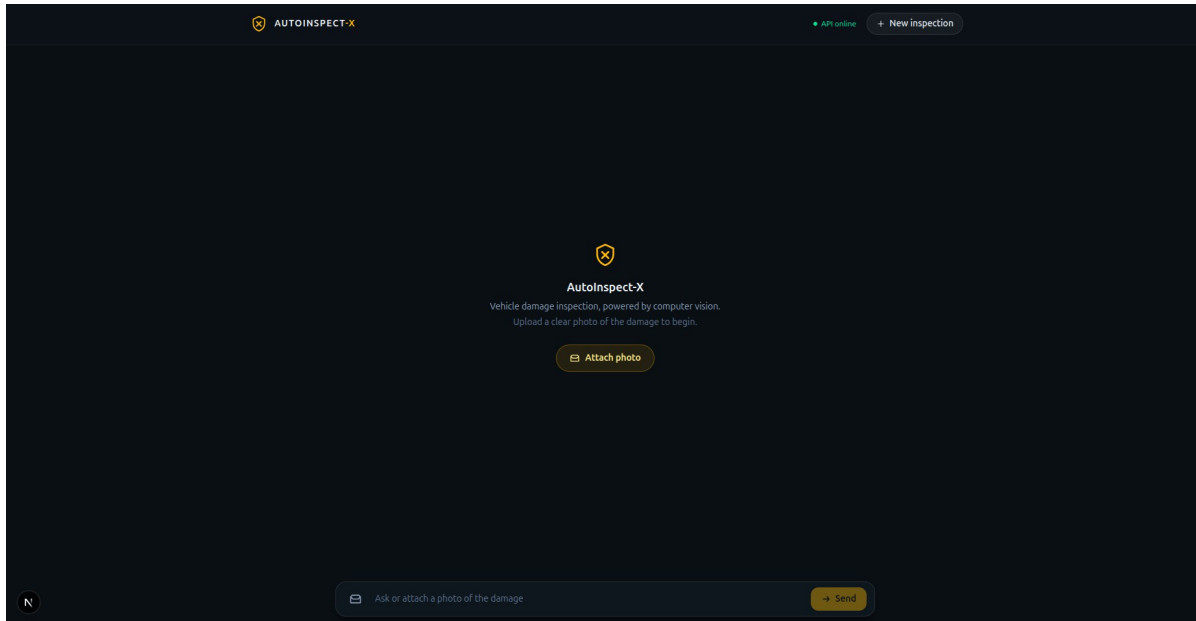


Fig. 5.3 Inspection composer at /demo with upload prompt and free-text field.

The quality gate was exercised with a deliberately blurred photograph (Fig. 5.4). The validator measured sharpness_variance 1.22, mean_luminance 139.28, bright_fraction 0.0 and contrast_std 74.08 – dominated by the sharpness failure against the 25.0 threshold – and the endpoint returned `QUALITY_FAILED` with retake guidance, producing no mask (Fig. 5.5).



Fig. 5.4 Blurred input photograph used to exercise the capture-quality gate.

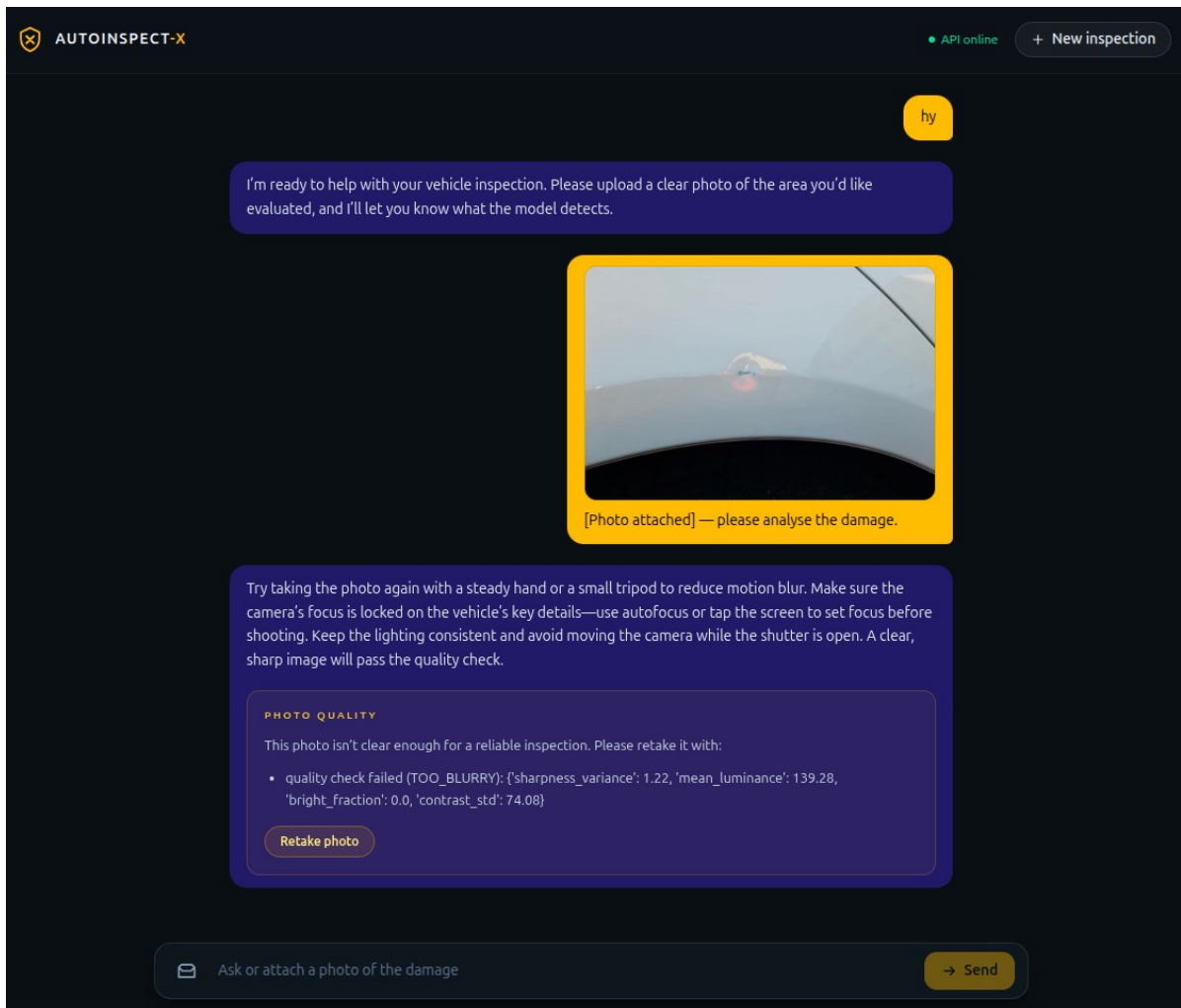


Fig. 5.5 Capture-quality rejection (TOO_BLURRY) with retake guidance; no mask is produced. Five valid photographs then exercised the full pipeline: a minor scratch (Figs. 5.6–5.7), a minor dent/scratch (Figs. 5.8–5.9), a medium multi-damage scene (Figs. 5.10–5.11), a major dent (Figs. 5.12–5.13) and a multi-cue (“confusing”) photo (Figs. 5.14–5.15).



Fig. 5.6 Minor-scratch test photograph (street scene).

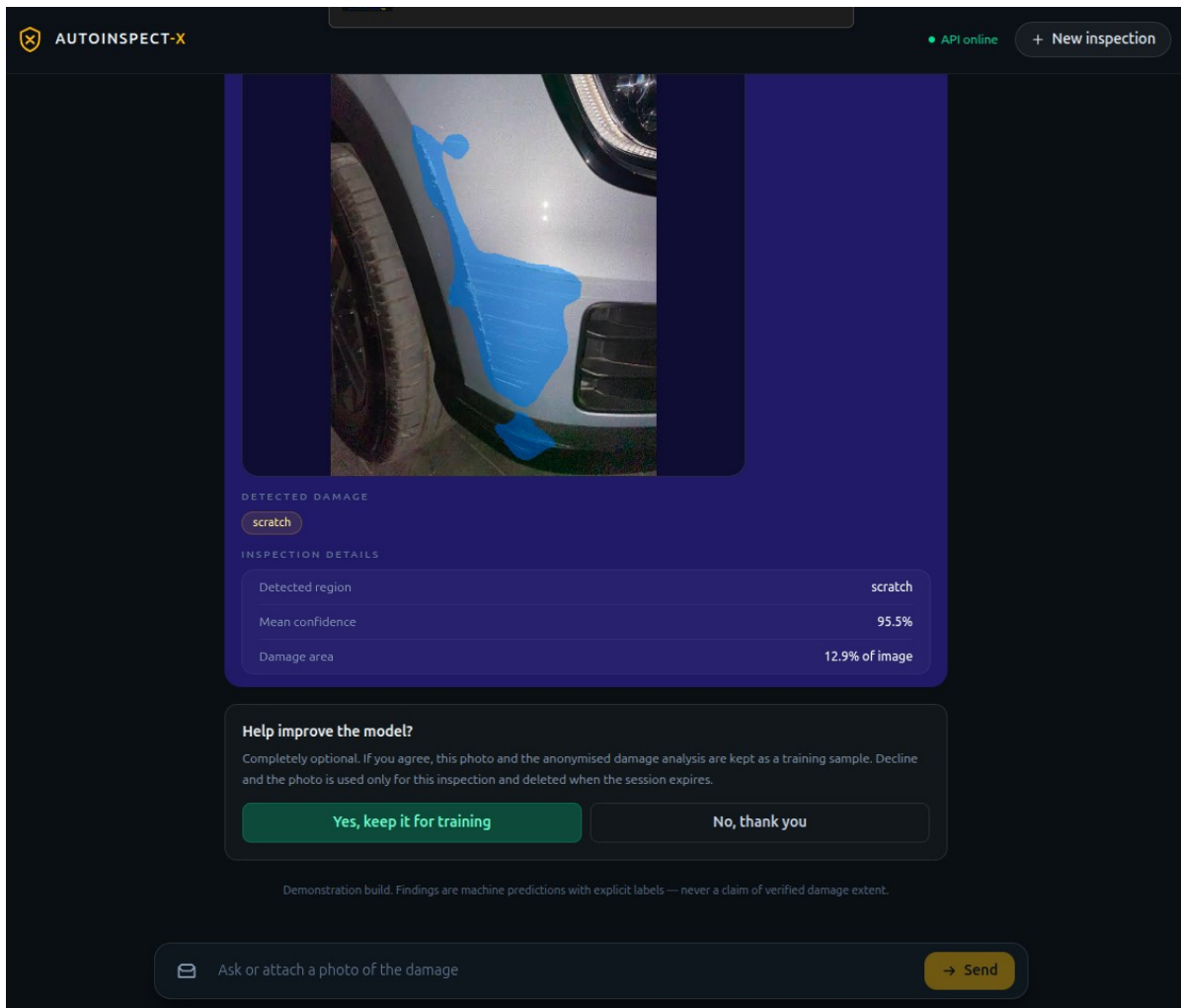


Fig. 5.7 Minor-scratch result - detected region, mean confidence, image-area fraction, consent prompt.



Fig. 5.8 Minor dent/scratch test photograph (close-up).

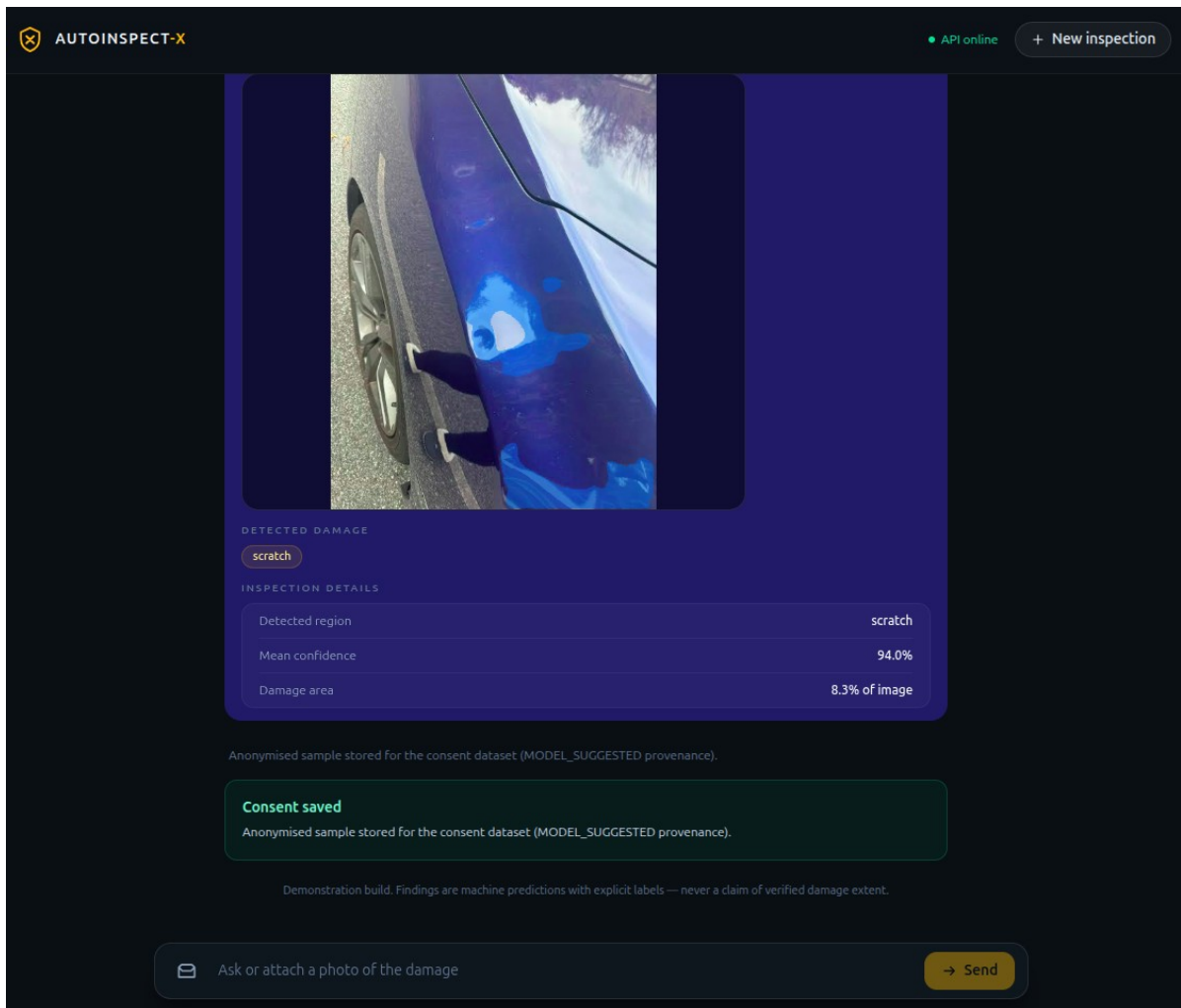


Fig. 5.9 Minor-damage result - scratch chip with confidence, damage-area ratio, consent granted.



Fig. 5.10 Medium-damage test photograph.

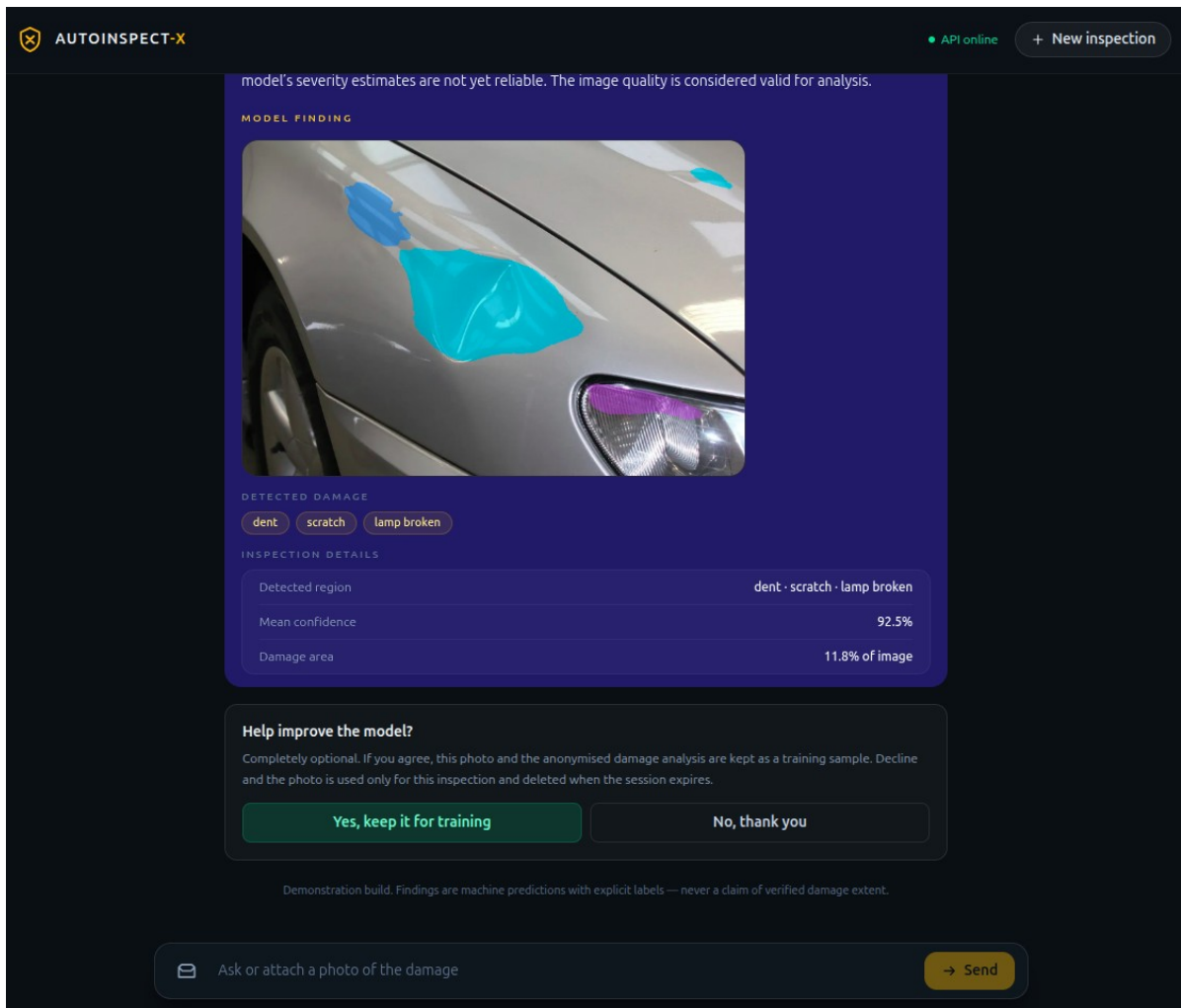


Fig. 5.11 Medium-damage result - dent, scratch and lamp-broken chips with consent prompt.



Fig. 5.12 Major-damage test photograph.

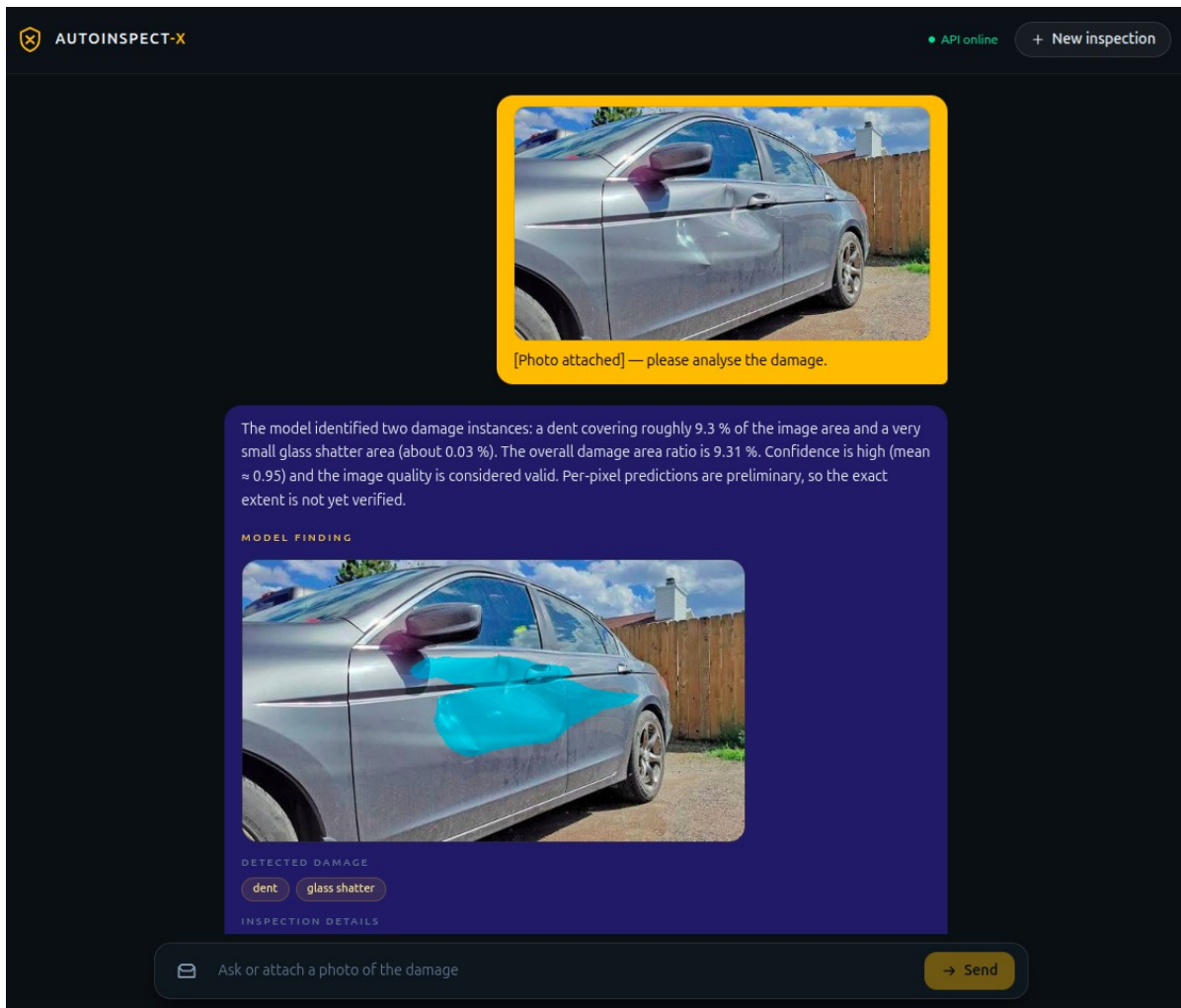


Fig. 5.13 Major-damage result - dent with a small glass-shatter region and consent prompt.



Fig. 5.14 Multi-cue (confusing) test photograph.

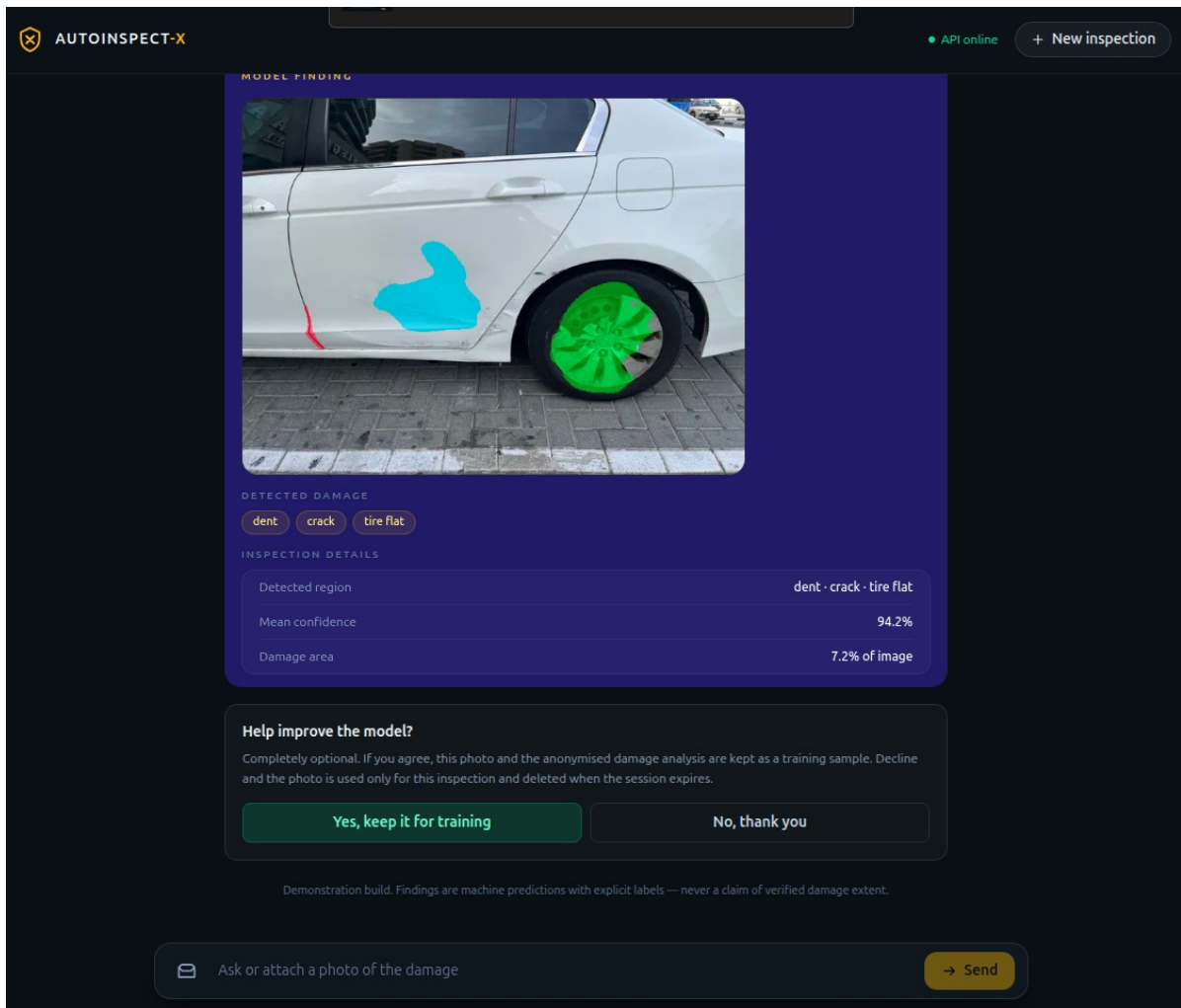


Fig. 5.15 Multi-cue result - dent, crack and tire-flat chips with mean confidence.

The observed outputs are summarised in Table 5.1 (blur rejection) and Table 5.2 (analysis scenes).

Metric	Measured	Threshold
sharpness_variance	1.22	min 25.0 (failed)
mean_luminance	139.28	min 45.0 (ok)
bright_fraction	0.00	max 0.30 (ok)
contrast_std	74.08	min 12.0 (ok)
status	TOO_BLURRY	QUALITY_FAILED → guidance only

Table 5.1 — capture-quality outcome for the blurred photo (on-screen values).

Scene	Classes detected	Damage area (Σ of image)	Mean conf.	Notes
MinorScratch	scratch	12.9%	95.5%	consent offered
minor dent	scratch (4 instances)	8.3%	94.0%	consent granted; sample saved
medium dent	dent, scratch, lamp	11.8% (8.9/1.5/1.4)	92.5%	valid; consent

	broken (4 inst.)			offered
Major dent	dent, glass shatter	9.31% (9.3/0.03)	94.8–95.0%	re-analysis identical
Confusing	dent, crack, tire flat	7.2% (2.9/0.12/4.2)	94.2%	quality VALID

Table 5.2 — demonstration analysis outcomes (values as rendered by the assistant).

Note how the assistant’s narration stays inside the evidence: it repeats area ratios and confidence, and it appends the disclaimer that masks are preliminary and severity estimates are not yet reliable. The consent flow shows “granted – sample stored (MODEL_SUGGESTED provenance)” for the minor-dent scene and an optional decline path elsewhere (additional screenshots in Appendix A).

5.4 SUMMARY

The prototype is complete to the product’s Phase R: a working photo-first demo with honest labels, offline-safe assistant, consent pipeline, end-to-end tests, Playwright suites and CI quality gates. Two deliberate engineering rules are enforced by tests – no cost/repair/quote field anywhere, and no session photo may be used for training without consent.

CHAPTER 6 PROJECT OUTCOME AND APPLICABILITY

6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

The measurable core of the project is the pair of segmentation checkpoints trained for the comparison: a ResNet-34 U-Net baseline (pilot15_baseline) and the CNN+Transformer hybrid (pilot15_hybrid). Both were trained on the full CarDD train split for 15 epochs at seed 0 with identical hyperparameters, so the only difference is the architecture (plus the extra bottleneck-transformer parameters of the hybrid). Everything around them – quality gate, features, assistant, consent, UI, tests – is already described in Chapter 5.

6.2 SIGNIFICANT PROJECT OUTCOMES

Table 6.1 records the locked training configuration (from the run records).

Configuration	pilot15_baseline	pilot15_hybrid
Model	ResNet34UNet (pretrained encoder)	HybridSegmentation (ResNet34 + bottleneck Transformer)
Parameters	24,577,757	27,933,661
Image size / batch	512×512 / effective 4	512×512 / effective 4
Optimizer / LR	AdamW; backbone 1e-4, head 3e-4	AdamW; backbone 1e-4, head 3e-4
Schedule	5-epoch warmup + cosine → 1e-6	5-epoch warmup + cosine → 1e-6
Loss	0.5 Dice + 0.5 Focal ($\gamma=2$), deep-sup 0.75/0.15/0.10	same
Class weights	inverse-frequency (fg); background 0.1	same
Regularization	EMA 0.999, weight decay 1e-4, mixed precision	same
Data	train-only augmentation: h-flip, affine (rot $\pm 8^\circ$, scale 0.9–1.1), brightness/contrast	same
Peak VRAM	≈ 1.89 GB	≈ 2.01 GB
Batches / epoch	704	704

Table 6.1 — locked training configuration (source: ml/experiments/*/run_record.json).

Table 6.2 reports the best validation metrics; both models peaked at epoch 14 and no early-stopping was triggered.

Val metric (epoch 14)	pilot15_baseline	pilot15_hybrid
Foreground mIoU	0.6127	0.5963
Foreground mDice	0.7440	0.7320
Pixel accuracy	0.8979	0.8873

Table 6.2 — best validation metrics (foreground = damage classes only; EMA checkpoint).

6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

Class	pilot15_baseline mIoU	pilot15_hybrid mIoU
background	0.8782	0.8659
dent	0.4803	0.4478
scratch	0.5480	0.5308
crack	0.3560	0.3533
glass shatter	0.9274	0.8874
lamp broken	0.6288	0.6511
tire flat	0.7357	0.7075

Table 6.3 — per-class validation mIoU at the best epoch.

Readings: (i) minority classes are the hardest – crack (651 training instances) lags at ≈ 0.35 mIoU in both models, while the frequently-sampled glass-shatter class exceeds 0.89; (ii) the hybrid trails the baseline by a small margin at this short training budget (about 0.016 mIoU), which is unsurprising for an attention module that has had only 15 epochs of training and no ablation-driven budget allocation; (iii) the figures are deliberately reported as intermediate. The model card shipped inside the engine therefore states foreground mIoU ≈ 0.60 – 0.61 measured at 15 epochs with several minority classes not reliably separated.

6.4 INFERENCE

The project outcomes are applicable in several near-term real-world contexts: first-party photo triage for insurance (deciding which claims deserve a desk review before a human adjuster or garage inspection); rental-car return documentation with a photographic, labelled damage record; workshop intake where the honest quality-gate-rejected photo prompts the technician to ask for a better picture rather than guess; and vision-training-data collection in which, with consent, everyday inspections become ground truth. The consent pipeline is the flywheel: the subset of samples explicitly offered by users is stored separately from session data and is ready to be passed through human verification before joining any future training set (provenance MODEL_SUGGESTED).

CHAPTER 7 CONCLUSIONS AND RECOMMENDATION

7.1 OUTLINE

The project delivered a working, honest, photo-first vehicle damage inspection system: a capture-quality gate, a CNN+Transformer hybrid segmentation model compared with a ResNet-34 U-Net baseline on CarDD, evidence-labelled features, a grounded conversational assistant, an opt-in consent pipeline, a polished Next.js demo and CI quality gates. Where the literature and common tools present damage percentages as facts and repair costs as predictions, this system prints “machine prediction, preliminary, not verified damage extent” and refuses cost questions by design – a contract enforced by tests.

7.2 LIMITATION/CONSTRAINTS OF THE SYSTEM

Constraints and limitations are documented honestly throughout: the 15-epoch pilots are intermediate evidence, not a final research conclusion, and the hybrid has not yet been given a fair training budget; minority classes (crack, small dent instances) are not reliably separated; damage area is image-denominator only and physically calibrated cm^2 is unavailable without a calibration method; the demonstration photographs are illustrative scenes, not a validated external test set, and a leakage-free external bench is planned but missing; the LLM depends on Groq availability (the stub is the offline fallback) and can still paraphrase evidence inaccurately even with grounding; Live inference runs on a single 4 GB laptop GPU; and the cinematic narration still contains one legacy cost line that awaits visual reconciliation under ADR 0011.

7.3 FUTURE ENHANCEMENTS

Recommended future work: (1) give the hybrid a real training budget (longer runs, ablations on transformer blocks/heads, warm-starting from the baseline) and re-run the comparison; (2) build the planned external, leakage-free validation set with verified labels; (3) add geometric calibration so a calibrated physical area becomes defensible where conditions allow; (4) human-in-the-loop verification of consented samples to lift MODEL_SUGGESTED provenance; (5) multi-view and temporal hints for occluded damage; (6) deployment hardening (container images, model registry, observability) and a production consent/privacy review; and (7) an accessibility and localisation pass on the demo.

7.4 INFERENCE

Inference: the photo-first contract is sound and implementable today. An honest-quality system – gate first, segment with a tractable model, label every number as a derived, not measured, value, ground the assistant, and ask before keeping the data – beats a flashier one that fabricates certainty. The foundation laid in Phase-I is the correct base for the longer training runs, external validation and human-in-the-loop refinement that define Phase-II.

APPENDIX A SCREENSHOTS

This appendix reproduces the supplementary browser captures. Figures 5.1–5.15 in Chapter 5 already show the primary screens; the figures here show the composer pre-analysis state and the assistant narratives and consent outcomes for each scene, without duplicating any figure used earlier.

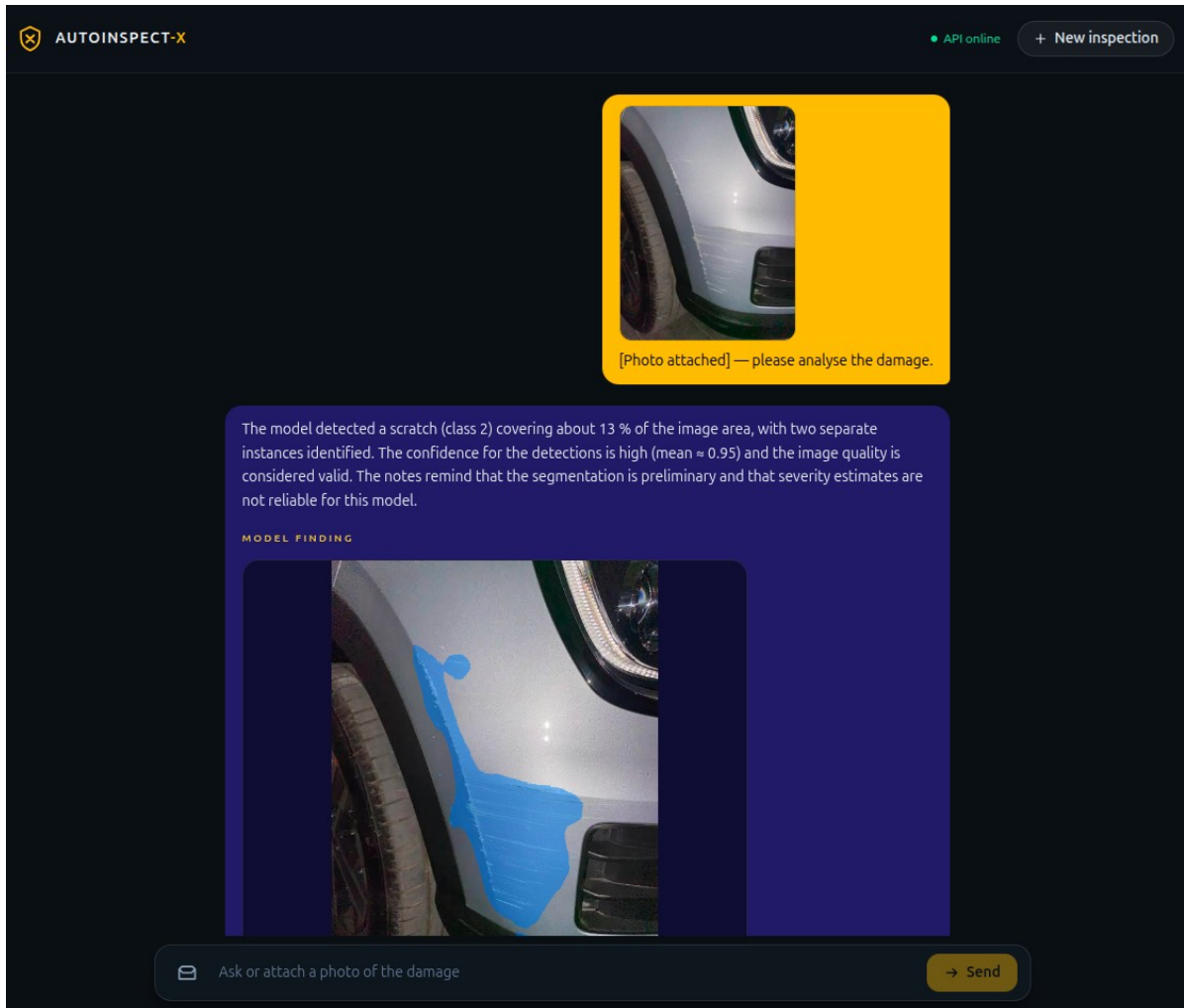


Fig. A.1 Composer with the photo attached before analysis (minor-scratch session).

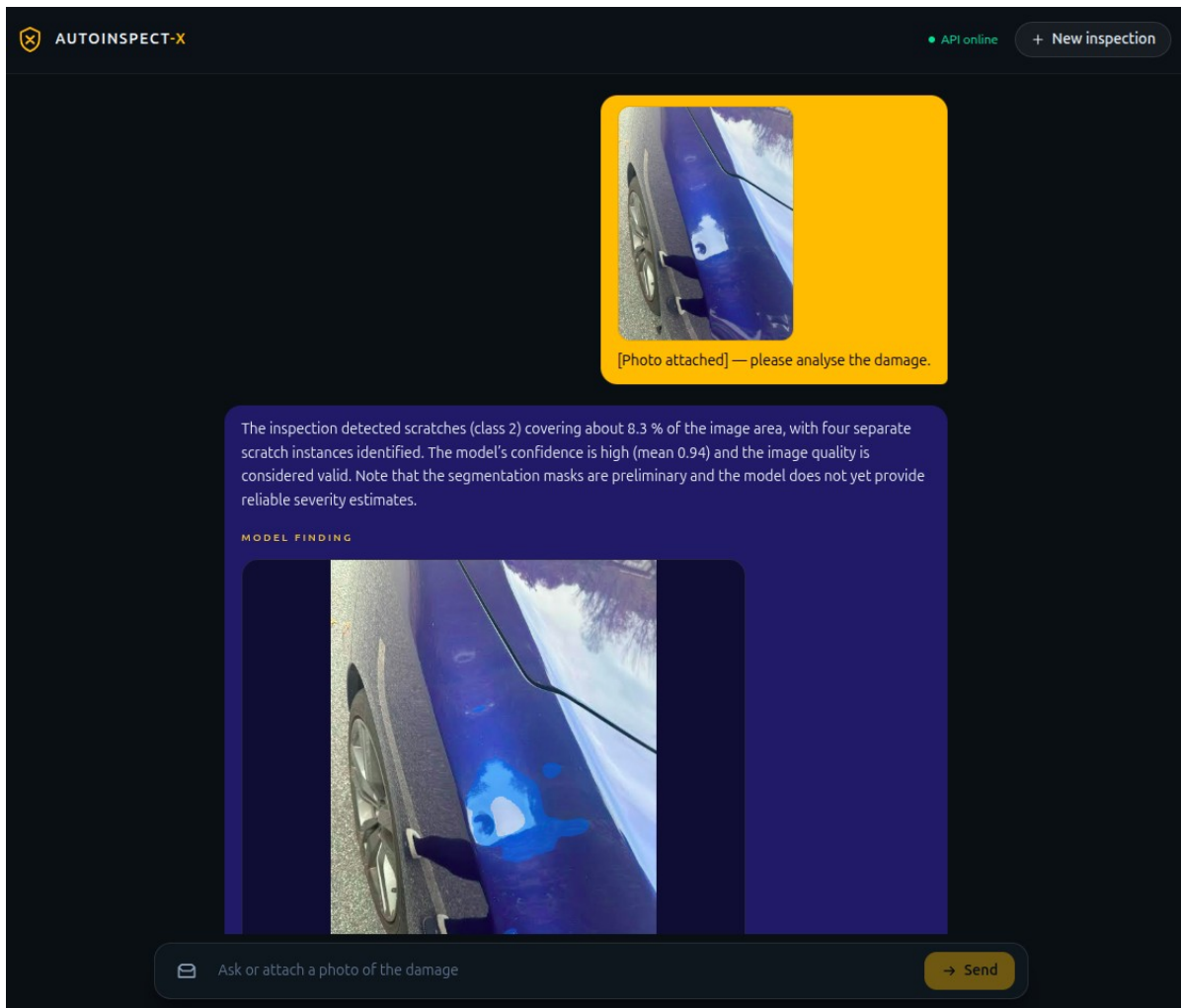


Fig. A.2 Assistant narrative for a minor dent (the assistant's “Here is what the visual model identified” summary).

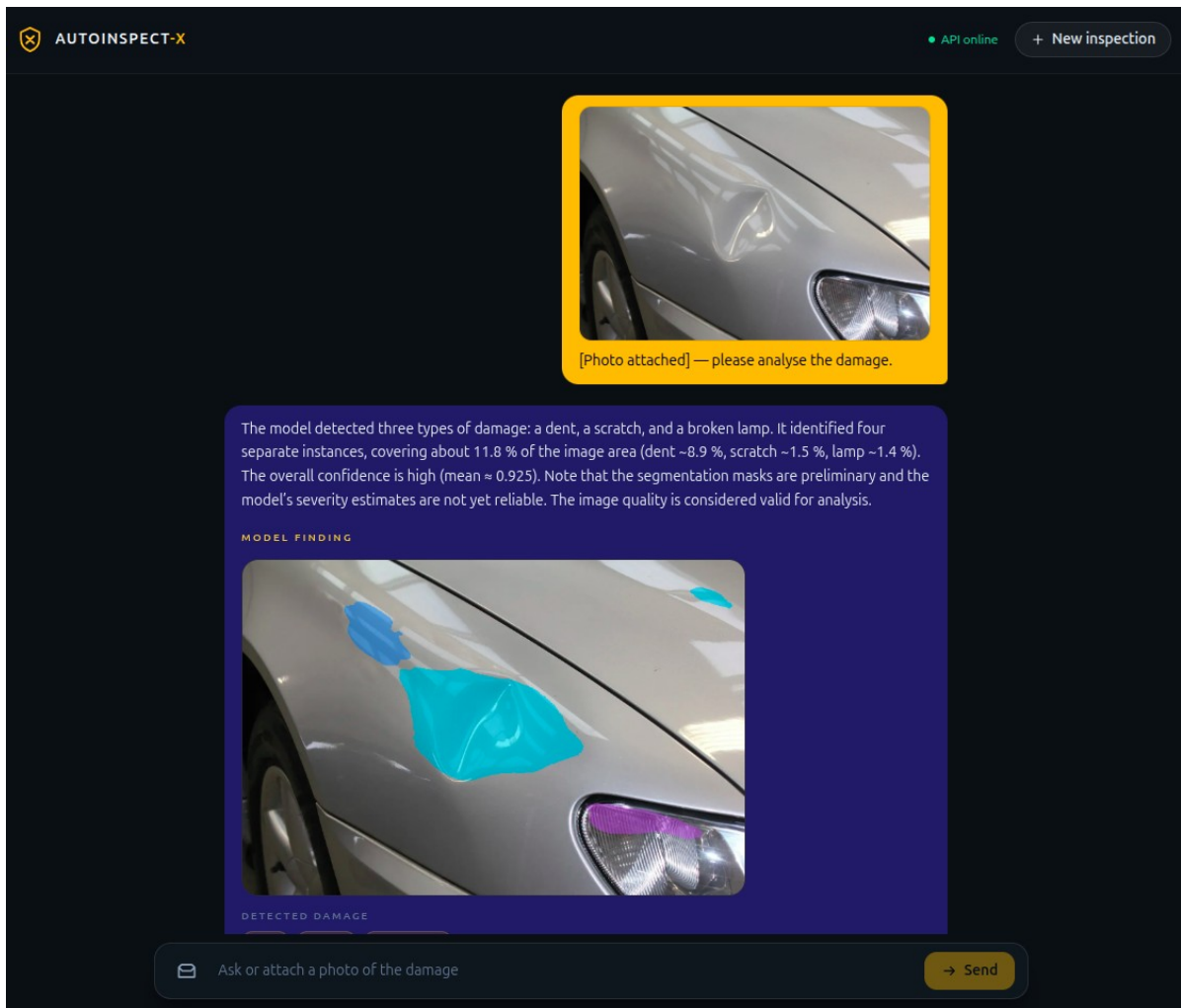


Fig. A.3 Assistant narrative for a medium-damage photograph.

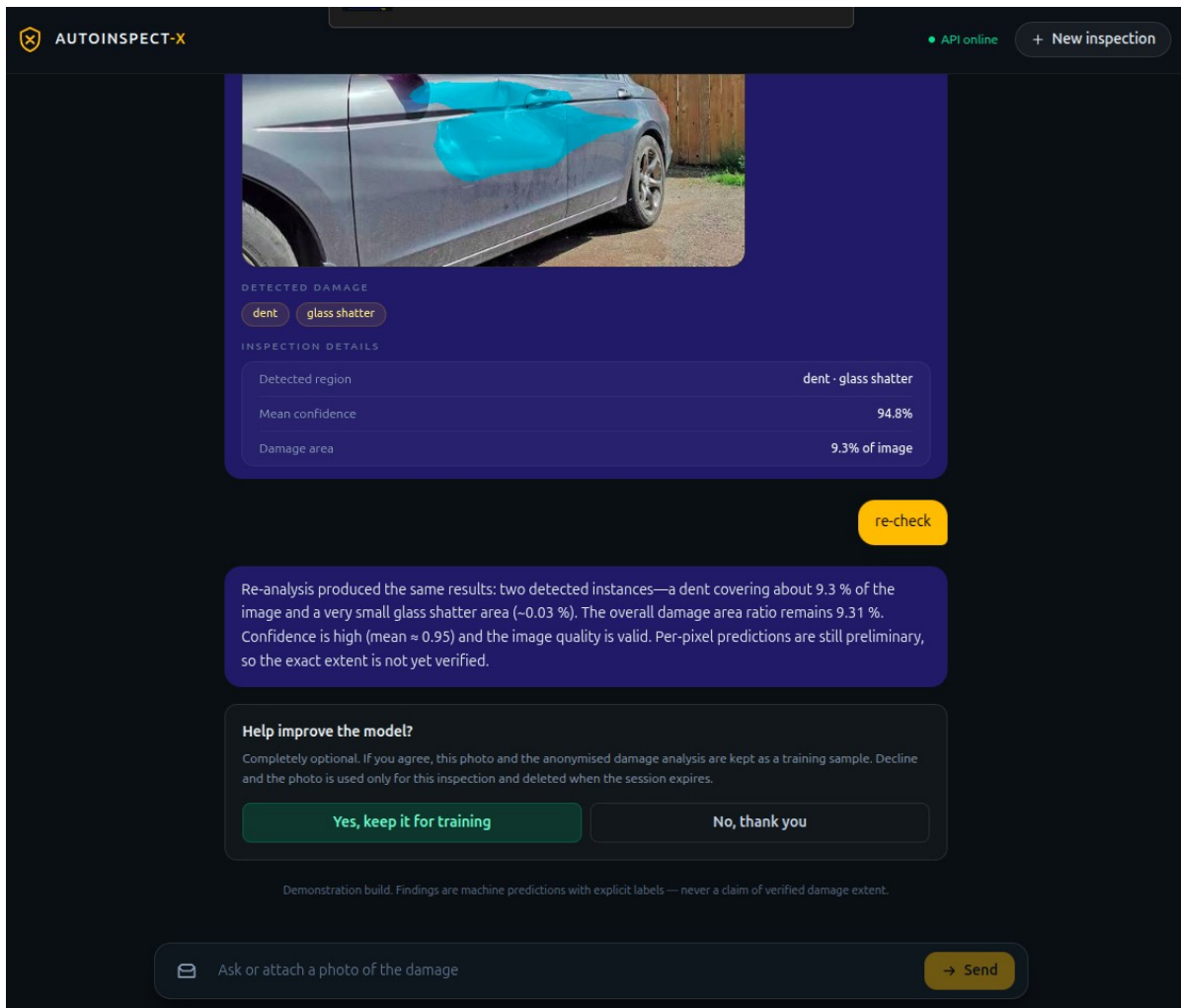


Fig. A.4 Major-damage result with consent granted and re-analysis confirmation.

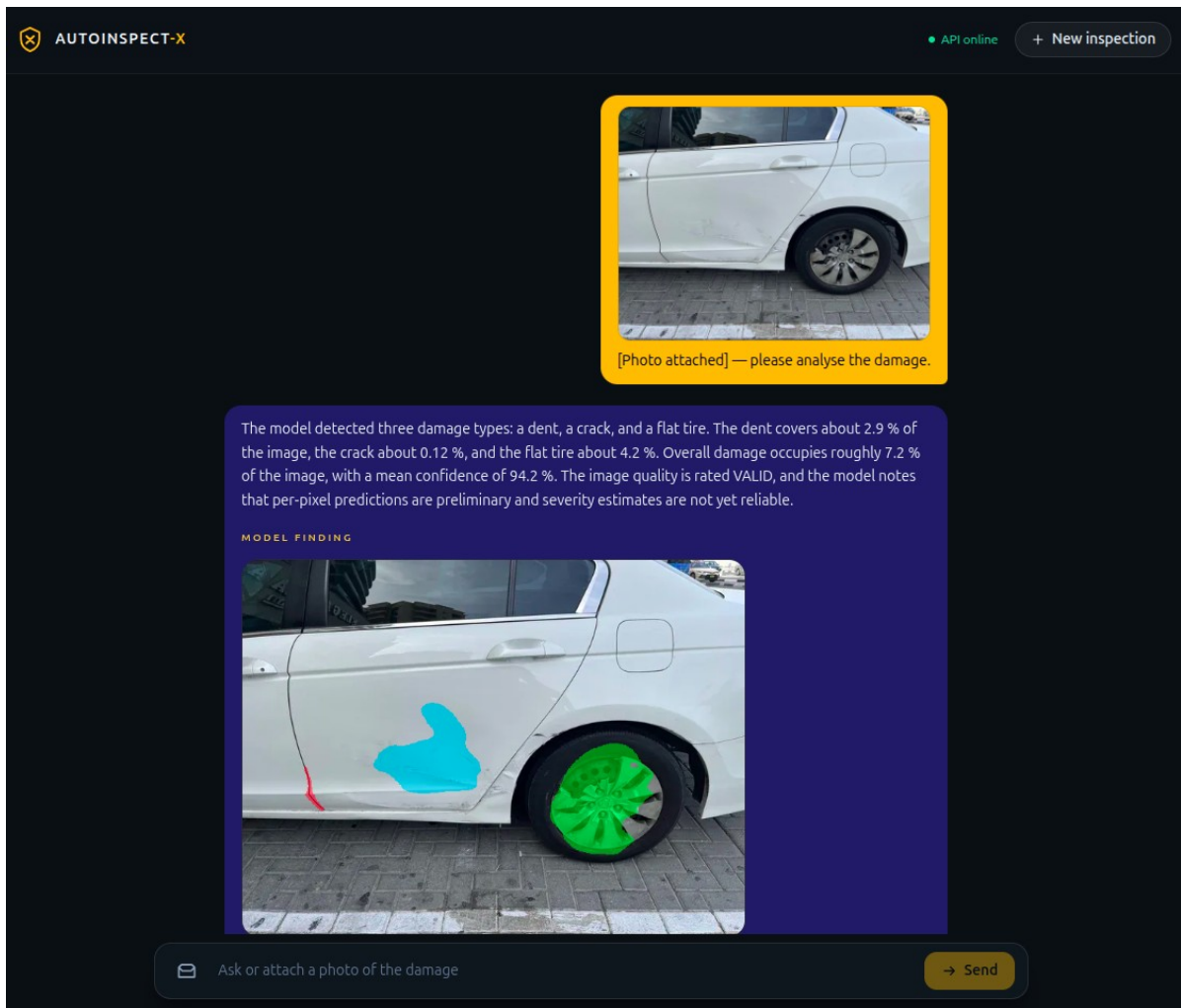


Fig. A.5 Assistant narrative for the multi-cue (confusing) photograph.

APPENDIX B CODE LISTINGS

Representative source code reproduced from the repository, grouped by layer. Each listing is a verbatim excerpt, abridged where noted; longer monospace blocks are kept on single-spacing to preserve line structure.

Listing B.1 — damage feature extraction (source: ml/inference/features.py, abridged).

Listing B.1

```
for class_id in range(1, int(result.prob.shape[0])):
    class_mask = (mask == class_id).astype(np.uint8)
    _, labels, stats, _ = cv2.connectedComponentsWithStats(class_mask, 8)
    for label in range(1, int(stats.shape[0])):
        x, y, bw, bh, area = (int(v) for v in stats[label][:5])
        if area < min_component_pixels:
            continue
        comp = labels == label
        weights = pixel_confidence[comp]
        instances.append(DamageInstance(
            instance_id=len(instances), class_id=class_id,
            area_pixels=area,
            area_ratio_image=round(area / image_pixels, 6),
            mean_confidence=round(float(weights.mean()), 4), ...))
# damage_pixels over the 512x512 denominator: damage_area_ratio_image ...
```

Listing B.2 — engine architecture dispatch (source: ml/inference/engine.py, abridged).

Listing B.2

```
def _build_model(arch: str, *, base: int, num_classes: int):
    arch_l = "".join(arch.lower().split("_"))
    if arch_l in {"resnet34unet", "baseline"}:
        return ResNet34UNet(num_classes=num_classes, pretrained=False)
    if arch_l in {"hybrid", "hybridsegmentation"}:
        return HybridSegmentation(num_classes=num_classes, pretrained=False)
    if arch_l in {"carddhybrid"}:
        return CarddHybrid(base=base, num_classes=num_classes)
    raise ModelVersionError(f"unknown model_arch in checkpoint: {arch!r}")
```

Listing B.3 — SQLite schema (source: apps/api/storage/database.py).

Listing B.3

```
CREATE TABLE IF NOT EXISTS sessions (
    id TEXT PRIMARY KEY, status TEXT NOT NULL, created_at TEXT NOT NULL,
    expires_at TEXT);
CREATE TABLE IF NOT EXISTS image_assets (
    id TEXT PRIMARY KEY, session_id TEXT NOT NULL, kind TEXT NOT NULL,
    relative_path TEXT NOT NULL, created_at TEXT NOT NULL,
    FOREIGN KEY (session_id) REFERENCES sessions(id) ON DELETE CASCADE);
-- Consented training side lives in separate tables so cleanup can never reac...
CREATE TABLE IF NOT EXISTS consent_records (
    id TEXT PRIMARY KEY, session_id TEXT NOT NULL, decision TEXT NOT NULL,
    dataset_version TEXT, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS training_samples (
    id TEXT PRIMARY KEY, session_id TEXT NOT NULL, consent_id TEXT NOT NULL,
    dataset_version TEXT NOT NULL, annotation_status TEXT NOT NULL,
    provenance TEXT NOT NULL, image_asset_id TEXT NOT NULL,
    labels_json TEXT NOT NULL, created_at TEXT NOT NULL);
CREATE TABLE IF NOT EXISTS session_states (
    id TEXT PRIMARY KEY, state_json TEXT NOT NULL, updated_at TEXT NOT NULL);
```

Listing B.4 — LangGraph turn workflow (source: apps/api/agent/graph.py, abridged).

Listing B.4

```
def llm_turn(state, services):
    reply = services.assistant.chat_reply(
        history=history_from_state(state),
        evidence=persist_evidence(dict(state)))
    return {"reply": reply}

def build_workflow(services):
    b = StateGraph(InspectionState)
    b.add_node("llm_turn", lambda s: llm_turn(s, services))
    b.add_edge(START, "llm_turn"); b.add_edge("llm_turn", END)
    return b.compile()
```

Listing B.5 — deep-supervised Dice+Focal (source: ml/training/loss.py, abridged).

Listing B.5

```
def level_loss(logits, targets, weights, smooth=1e-6, gamma=2.0):
    targets_idx = targets.argmax(dim=1)
    dice = _soft_dice_loss(logits, targets, smooth=smooth)
    focal = _focal_loss(logits, targets_idx, weights, gamma=gamma)
    return 0.5 * dice + 0.5 * focal

def dice_focal_loss(main, aux1, aux2, masks, labels, num_classes, weights):
    targets = aggregate_targets(masks, labels, num_classes)
    total = 0.75 * level_loss(main, targets, ...)
    if aux1 is not None: total = total + 0.15 * level_loss(aux1, ...)
    if aux2 is not None: total = total + 0.10 * level_loss(aux2, ...)
    return total
```

Listing B.6 — bottleneck transformer (source: ml/models/hybrid_segmentation.py, abridged).

Listing B.6

```
class BottleneckTransformer(nn.Module):
    # Input [B,512,16,16] -> project 1x1 to 256 -> 256 tokens -> [B,256,16,16]
    def __init__(self, in_channels=512, d_model=256, num_blocks=4,
                 num_heads=4, ffn_dim=1024, dropout=0.10):
        self.project = nn.Conv2d(in_channels, d_model, kernel_size=1)
        self.pos_embed = nn.Parameter(torch.zeros(1, 256, d_model))
        self.blocks = nn.TransformerEncoder(
            nn.TransformerEncoderLayer(d_model, nhead=num_heads,
                                       dim_feedforward=ffn_dim, dropout=dropout,
                                       activation="gelu", norm_first=True, batch_first=True),
            num_layers=num_blocks)
    def forward(self, c5):
        x = self.project(c5)
        tokens = x.flatten(2).transpose(1, 2) + self.pos_embed
        tokens = self.blocks(tokens)
        return tokens.transpose(1, 2).reshape(batch, ch, h, w)
```

Listing B.7 — assistant honesty contract (source: apps/api/agent/assistant.py, abridged).

Listing B.7

```
_PRIMARY_SYSTEM_PROMPT = (
    "You are a professional automotive damage inspection assistant ...
    RULES:
    - Describe ONLY what the structured inspection evidence says.
    - NEVER invent damage, severity, repair actions, costs, or part prices.
    - Cost and repair pricing are out of scope; if asked, decline briefly.
    - Evidence flagged low_confidence is preliminary - say so.")

class StubAssistant: # deterministic offline double, provider="stub"
    def damages_explanation(self, evidence):
        # echoes only evidence classes/ratios; offline stand-in, never a live...
        ...
```

Listing B.8 — typed client (source: apps/web/lib/api.ts, abridged).

Listing B.8

```
const API_BASE = process.env.NEXT_PUBLIC_API_URL ?? "http://localhost:8000";
export async function uploadPhoto(sessionId, file) {
  const form = new FormData(); form.append("file", file);
  return request(`/inspection/${sessionId}/upload`, {method:"POST", body: for...
}
export function analyzePhoto(sessionId) {
  return request(`/inspection/${sessionId}/analyze`, {method:"POST"});
}
export function sendConsent(sessionId, decision) {
  return request(`/inspection/${sessionId}/consent`, {method:"POST",
    body: JSON.stringify({ decision })});
}
```

Listing B.9 — Google Colab (source: AutoInspectX_Final_Training.ipynb).

Listing B.9

```
import torch
print("PyTorch:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
    print("VRAM:", round(torch.cuda.get_device_properties(0).total_memory /
1024**3, 2), "GB")
else:
    raise RuntimeError("CUDA GPU is required for final training.")

%cd /content/Coding/college/Capstone1
!mkdir -p ml/experiments
!mkdir -p /content/drive/MyDrive/AutoInspectX/experiments/final60_baseline_seed42
!rm -rf ml/experiments/final60_baseline_seed42
!ln -s \
    /content/drive/MyDrive/AutoInspectX/experiments/final60_baseline_seed42 \
    ml/experiments/final60_baseline_seed42
!ls -ld ml/experiments/final60_baseline_seed42

%cd /content/Coding/college/Capstone1
!python -m ml.training.train \
    --data-root /content/data/CarDD_COCO \
    --model resnet34_unet \
    --epochs 60 \
    --batch-size 4 \
    --grad-accum 1 \
    --num-workers 2 \
    --seed 42 \
    --checkpoint-every 5 \
    --label final60_baseline_seed42
```

```

from pathlib import Path
exp =
Path("/content/drive/MyDrive/AutoInspectX/experiments/final60_baseline_seed42")
print("Experiment files:")
for p in sorted(exp.glob("*")):
    print(p.name)

import torch
from pathlib import Path
exp =
Path("/content/drive/MyDrive/AutoInspectX/experiments/final60_baseline_seed42")
for name in ["best_checkpoint.pt", "periodic_epoch_0004.pt"]:
    path = exp / name
    ckpt = torch.load(path, map_location="cpu", weights_only=False)
    print("\n" + "=" * 60)
    print(name)
    print("=" * 60)
    for key, value in ckpt.items():
        if key in {
            "model_state",
            "ema_state",
            "optimizer_state",
            "scaler_state",
        }:
            print(f"{key}: present")
        elif key == "config":
            print(f"config.epochs: {value.get('epochs')}")
            print(f"config.label: {value.get('experiment_label')}")
        else:
            print(f"{key}: {value}")

%cd /content/Coding/college/Capstone1

# =====
# TRAINING CONFIG – CHANGE ONLY THESE VALUES
# =====

MODEL = "resnet34_unet"          # "resnet34_unet" or "cnn_transformer"
LABEL = "final60_baseline_seed42"
EPOCHS = 60
BATCH_SIZE = 4
GRAD_ACCUM = 1
NUM_WORKERS = 2
SEED = 42

```

```

# Optional resume checkpoint.
# Use None for a fresh run.
# RESUME = None
RESUME =
"/content/drive/MyDrive/AutoInspectX/experiments/final60_baseline_seed42/
best_checkpoint.pt"

# =====
# DO NOT CHANGE BELOW THIS LINE
# =====

cmd = [
    "python", "-m", "ml.training.train",
    "--data-root", "/content/data/CarDD_COCo",
    "--model", MODEL,
    "--epochs", str(EPOCHS),
    "--batch-size", str(BATCH_SIZE),
    "--grad-accum", str(GRAD_ACCUM),
    "--num-workers", str(NUM_WORKERS),
    "--seed", str(SEED),
    "--checkpoint-every", "5",
    "--label", LABEL,
]

if RESUME is not None:
    cmd += ["--resume", RESUME]
print("Training configuration:")
print("  Model:", MODEL)
print("  Label:", LABEL)
print("  Epochs:", EPOCHS)
print("  Batch size:", BATCH_SIZE)
print("  Grad accumulation:", GRAD_ACCUM)
print("  Effective batch:", BATCH_SIZE * GRAD_ACCUM)
print("  Workers:", NUM_WORKERS)
print("  Seed:", SEED)
print("  Resume:", RESUME)
print("\nStarting training...\n")
!{" ".join(cmd)}

```


REFERENCES

- [1] O. Ronneberger, P. Fischer and T. Brox, “U-Net: Convolutional Networks for Biomedical Image Segmentation,” in MICCAI, 2015.
- [2] K. He, X. Zhang, S. Ren and J. Sun, “Deep Residual Learning for Image Recognition,” in CVPR, 2016.
- [3] A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” in ICLR, 2021 (arXiv:2010.11929).
- [4] X. Wang, W. Li, Z. Wu, ... , “CarDD: A New Dataset for Vision-Based Car Damage Detection,” IEEE Transactions on Intelligent Transportation Systems, 2023. DOI 10.1109/TITS.2023.3258480.
- [5] T. Huynh et al., “VehiDE: A Dataset for Vehicle Damage ... ,” in KSE, 2023.
- [6] J. B. Parsløv et al., “CrashCar101: Crash severity and damage localisation ... ,” in WACV, 2024.
- [7] B. Bhogale, S. Mandal, A. Tiwary and A. Somakumar, “CDA-Net: Camera and depth attention network for damage detection in vehicles,” 2023.
- [8] T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, “Focal Loss for Dense Object Detection,” in ICCV, 2017.
- [9] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in NeurIPS, 2019.
- [10] LangChain & LangGraph documentation, <https://python.langchain.com> and <https://langchain-ai.github.io/langgraph/>.
- [11] FastAPI documentation, <https://fastapi.tiangolo.com>.
- [12] Next.js documentation, <https://nextjs.org/docs>.
- [13] React documentation, <https://react.dev>.
- [14] Playwright documentation, <https://playwright.dev>.
- [15] OpenCV documentation, <https://docs.opencv.org>.
- [16] AutoInspect-X research report (project internal research document), 2026.
- [17] AutoInspect-X decision records ADR 0010 (hybrid model lineage) and ADR 0011 (photo-first scope; cost/repair/questionnaire removed), 2026.
- [18] AutoInspectX_Final_Training.ipynb, <https://colab.research.google.com>